

Securitize EVM On/Off Ramp Audit Report

Prepared by Cyfrin

Version 2.0

Head of Security Research

Dacian (Proprietary AI)

Lead Auditors

Dacian

November 12, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risks of Publishing This Report **2**

4 Audit Overview **3**

5 Audit Scope **4**

6 Summary of Findings **4**

7 Protocol Summary **5**

8 Executive Summary **5**
8.1 Centralization Risks 5

9 Findings **6**
9.1 Low Risk 6

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risks of Publishing This Report

This report may contain security-sensitive information beyond its findings. The executive summary, post-audit recommendations and other analysis can point to parts of the code that were hardest to review or carry the most residual risk. Attackers study published audit reports for exactly this information, and AI tools make it cheap to extract at scale.

Before publishing this report, Securitize should consider the potential for increased exploit risk. Cyfrin provides an audit summary for public use, which states the scope, commits, review period and finding counts without finding details. Where publication is required, for example for exchange listings or governance, the audit summary and its attestation may meet that need. If Securitize runs a bug bounty program, it can list acknowledged findings as known issues so they are not re-reported, without publishing the full report.

4 Audit Overview

Over the course of 1 days, the Cyfrin team conducted an audit on the EVM On/Off Ramp code provided by Securitize. The findings consist of 1 Low severity issue.

Summary

Project Name	EVM On/Off Ramp
Repository	bc-on-off-ramp-sc
Commit	2e785166d636...
Fix Commit	0a79f220ffc2...
Audit Timeline	Nov 12th - Nov 12th, 2026
Methods	Manual Review, Proprietary AI

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	1	1	0
Informational	0	0	0
Gas Optimizations	0	0	0
Total	1	1	0

How to read the statuses

Resolved means Cyfrin verified the remediation documented in this report with a Cyfrin verification note. Acknowledged means Securitize chose to retain the reported behavior or accept the residual risk; this does not constitute resolution or endorsement by Cyfrin. Finding-specific explanations and verification notes are recorded in this report.

Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

5 Audit Scope

The audit scope was limited to the following files:

```
contracts/common/BaseContract.sol
contracts/common/BaseExternalProvider.sol
contracts/common/BaseOnOffRamp.sol
contracts/fee/MbpsFeeManager.sol
contracts/off-ramp/provider/AllowanceLiquidityProvider.sol
contracts/off-ramp/provider/CollateralLiquidityProvider.sol
contracts/off-ramp/provider/ExternalLiquidityProvider.sol
contracts/off-ramp/BaseOffRamp.sol
contracts/off-ramp/CountryValidator.sol
contracts/off-ramp/ExternalLiquidityProviderOffRamp.sol
contracts/off-ramp/PublicStockOffRamp.sol
contracts/off-ramp/RedemptionManager.sol
contracts/off-ramp/RedemptionValidator.sol
contracts/off-ramp/SecuritizeOffRamp.sol
contracts/off-ramp/TokenCalculator.sol
contracts/on-ramp/provider/MintingAssetProvider.sol
contracts/on-ramp/provider/ExternalAssetProvider.sol
contracts/on-ramp/provider/AllowanceAssetProvider.sol
contracts/on-ramp/BaseOnRamp.sol
contracts/on-ramp/ExternalAssetProviderOnRamp.sol
contracts/on-ramp/PublicStockOnRamp.sol
contracts/on-ramp/SecuritizeOnRamp.sol
```

6 Summary of Findings

[L-1] `SecuritizeOnRamp::executePreApprovedTransaction` lets an EXCHANGE signer make any call as the ramp, handing EXCHANGE free minting, ISSUER role grants and theft of investor allowances	Resolved

7 Protocol Summary

`SecuritizeOnRamp` lets registered investors buy Securitize DS tokens, regulated tokenized securities such as BUIDL, with stablecoins, priced from a NAV rate less a configurable fee. Besides investors' own purchases, it relays subscriptions that a DS token `ISSUER` or `EXCHANGE` signer pre-approves off-chain:

- `executePreApprovedTransaction` checks an EIP-712 signature over the call and a per-investor nonce, recovers a signer holding the DS token's `ISSUER` or `EXCHANGE` role, and runs the call from the on-ramp's own address
- `subscribe`, callable only by the on-ramp itself, registers or updates the investor in the DS token registry, pulls the investor's stablecoins and delivers the purchased DS tokens to them
- `ExternalAssetProviderOnRamp` inherits the same relay

8 Executive Summary

8.1 Centralization Risks

- **DS token** `ISSUER` **or** `EXCHANGE` **signers** pre-approve each relayed `subscribe` call and choose its arguments -> could register an investor with any country and attributes, or spend stablecoins an investor approved to the on-ramp on a purchase delivered to that investor

9 Findings

9.1 Low Risk

9.1.1 `SecuritizeOnRamp::executePreApprovedTransaction` lets an EXCHANGE signer make any call as the ramp, handing EXCHANGE free minting, ISSUER role grants and theft of investor allowances

Description:

`SecuritizeOnRamp::executePreApprovedTransaction` accepts a signature from any trust-service EXCHANGE or ISSUER and then runs the signer-chosen call from the ramp's own address, with no restriction on `destination` or selector:

```
if (signerRole != trustService.EXCHANGE() && signerRole != trustService.ISSUER()) {
    revert InvalidEIP712SignatureError();
}
noncePerInvestor[txData.senderInvestor] = noncePerInvestor[txData.senderInvestor] + 1;
Address.functionCall(txData.destination, txData.data);
```

The intended use is a self-call into `SecuritizeOnRamp::subscribe`, but the ramp holds authority EXCHANGE does not. It is the only caller `MintingAssetProvider.supplyTo` accepts, so `supplyTo(attackerWallet, X)` mints X DS tokens for no liquidity (on `AllowanceAssetProvider` it moves the provider wallet's tokens). It is the spender of investors' liquidity-token allowances, so `transferFrom(investor, attacker, amount)` takes them. And `subscribe` needs the ramp to hold ISSUER or TRANSFER_AGENT for `updateInvestor` (DSToken 3.0.0 to 4.0.0), so the signer can also make the ramp issue, burn or seize tokens and grant itself a lasting ISSUER role. EXCHANGE's own DS powers are registry management only; a single malicious or compromised EXCHANGE key therefore exercises issuer-level and allowance-spending powers. `ExternalAssetProviderOnRamp` inherits the same relay.

Recommended Mitigation:

Accept only a self-call into `subscribe` in `executePreApprovedTransaction`, so a signature can no longer authorise any other call from the ramp's address:

```
--- a/contracts/on-ramp/SecuritizeOnRamp.sol
+++ b/contracts/on-ramp/SecuritizeOnRamp.sol
@@ -163,6 +163,10 @@
         bytes memory signature,
         ExecutePreApprovedTransaction calldata txData
     ) public whenNotPaused {
+        if (txData.destination != address(this) || bytes4(txData.data) != this.subscribe.selector) {
+            revert OnlySecuritizeOnRampError();
+        }
+
         bytes32 digest = hashTx(txData);
         address signer = ECDSA.recover(digest, signature);
```

A signer can still choose the arguments of that `subscribe` call, which is the relay's purpose.

Securitize:

Fixed at commit 0a79f22.

Cyfrin:

Verified. `SecuritizeOnRamp::executePreApprovedTransaction` now reverts with `InvalidDestinationError` unless `txData.destination` is the on-ramp itself and with `InvalidFunctionSelectorError` unless `txData.data` begins with the `subscribe` selector, and it always calls `address(this)`. A pre-approved signature can therefore only run `subscribe` on the on-ramp, so it can no longer call `MintingAssetProvider.supplyTo`, move an investor's liquidity-token allowance or use the on-ramp's DS token role beyond the investor registration `subscribe` performs. `ExternalAssetProviderOnRamp` inherits the check.